

Python Basics Cheatsheet

Scientific Programming 1

1. Basic Calculation Operations

```
5 + 3 # 8          5 / 3 # 1.66...
5 - 3 # 2          5 // 3 # 1
5 * 3 # 15         5 % 3 # 2
5 ** 3 # 125
```

```
# Rounding
round(1.6667) # 2
round(1.6667, 2) # 1.67
int(1.6667) # 1
```

Math

(you cannot use math functions during exam)

```
import math
```

```
# Constants
math.pi # 3.14159...
math.e # 2.718...
```

```
# Power and Roots
math.sqrt(16) # 4
math.pow(2, 3) # 8
math.exp(1) # e^1
math.log(10) # log(10) base e
math.log(100, 10) # log(10) base 10
```

Trigonometric Functions

```
math.sin(math.pi / 2) # 1
math.cos(0) # 1
math.tan(math.pi / 4) # 1
```

Inverse Trigonometric Functions

```
math.asin(1) # pi/2
math.acos(1) # 0
math.atan(1) # pi/4
```

Rounding

```
math.floor(3.7) # 3
math.ceil(3.2) # 4
```

3. Basic Type Conversions

```
int("42") # String to int
float("3.14") # String to float
str(42) # Int to string
list("abcd") # String to list
```

4. Lists

```
# Create a list
my_list = [3, 2, 1]
```

```
# Operations
item = my_list[2] # 1 - element at index 2
my_list.append(3) # [3, 2, 1, 3]
new_list = my_list + [5, 6] # [3, 2, 1, 3, 5, 6]
x = len(new_list) # 6 - length of list
"Apple" in new_list # False
```

```
# Looping 1
for item in my_list:
    print(item)
```

```
# Looping 2
for i in range(len(my_list)):
    item = my_list[i]
    print(item)
```

```
# Methods
new_list.sort() # [1, 2, 3, 3, 5, 6]
new_list.insert(2, 10) # [1, 2, 10, 3, 3, 5, 6]
new_list.remove(3) # [1, 2, 10, 3, 5, 6]
i = new_list.index(10) # 2 (↑ index of 10: 2)
```

```
# Slicing
lst = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
lst[2:5] # [2, 3, 4] (from 2 to 4)
lst[:3] # [0, 1, 2] (from start to 2)
lst[7:] # [7, 8, 9] (from 7 to end)
lst[-3:] # [7, 8, 9] (last 3)
lst[::-2] # [0, 2, 4, 6, 8] (every 2nd)
lst[1:8:3] # [1, 4, 7] (from 1 to 7 step 3)
lst[::-1] # [9, 8, ...] (reverses list)
lst[8:2:-2] # [8, 6, 4] (from 8 to 3 step -2)
```

5. Strings

```
# Strings
my_str = "Hello, World!"
```

```
my_str.lower() # Convert to lowercase
my_str.upper() # Convert to uppercase
my_str.capitalize() # Only first char uppercase
my_str.lstrip() # Remove leading whitespace
my_str.rstrip() # Remove trailing whitespace
my_str.strip() # Combined lstrip and rstrip
my_str.isalpha() # all characters alphabetic?
my_str.isdigit() # all characters digits?
my_str.islower() # all characters lowercase?
my_str.isupper() # all characters uppercase?
my_str.split(" ") # split string on spaces
```

```
# Looping through characters 1
for char in my_str:
    print(char)
```

```
# Looping through characters 2
for i in range(len(my_str)):
    char = my_str[i]
    print(char)
```

6. Reading files

```
# Open and read whole file
file = open("file.txt", "r")
content = file.read()
print(content)
file.close()
```

```
# Open and read line by line
file = open("file.txt", "r")
for line in file:
    print(line)
file.close()
```

```
# Alternative method: with open
with open("file.txt", "r") as file:
    for line in file:
        print(line)
```

Dictionaries, Sets and Classes

Scientific Programming 2

1. Dictionaries

```
# Create a dictionary: key -> value
ages = {"Ann": 28, "Bob": 24}

# Look up, add and change
ages["Ann"]          # 28
ages["Cem"] = 35      # add a new key
ages["Ann"] = 29      # change a value
ages["Dan"]          # KeyError!
ages.get("Dan", 0)    # 0 (default if missing)
"Bob" in ages         # True (checks keys)
len(ages)            # 3
del ages["Bob"]       # remove a key
ages.pop("Cem")       # remove, returns 35

# Looping 1: over the keys
for name in ages:
    print(name, ages[name])

# Looping 2: over key-value pairs
for name, age in ages.items():
    print(name, age)

list(ages.values())   # all values as a list
```

2. Sets

```
# Create a set: unique elements, no order
s = {3, 1, 4}
empty = set()          # not {}: that is a dict!
list(set(my_list))     # remove duplicates

# Operations
s.add(5)               # add (no duplicates)
s.remove(3)            # KeyError if missing
s.discard(9)           # no error if missing
4 in s                 # True
len(s)                 # 3
s[0]                   # TypeError: no index!

# Set operations
a = {1, 2, 3, 4}
b = {3, 4, 5}
a | b                  # {1, 2, 3, 4, 5} a.union(b)
a & b                  # {3, 4} a.intersection(b)
a - b                  # {1, 2} a.difference(b)
b.issubset(a)          # False
```

3. Tuples

```
# Create a tuple: like a list, but immutable
t = (42, "hello", 3.14)
t[1]                   # "hello"
len(t)                 # 3
t[1] = 9               # TypeError: immutable!
t.append(7)            # AttributeError!
tuple(my_list)         # list to tuple

# Unpacking
a, b, c = t             # a = 42, b = "hello", ...
a, b = t               # ValueError: too many
```

4. Classes

```
class MyClass:
    # Initializer: runs when creating an object
    def __init__(self, x):
        self.x = x

    # a method without parameters
    def method1(self):
        ...

    # a method with a parameters
    def method2(self, a):
        ...

    ...

r = MyClass(42)        # calls __init__ with x -> 42
r.method1()            # (method call)
r.method2("foo")       # (method call with parameter)

self is the object itself. A name starting with _, like
self._cards, is private to the class.
```

5. Inheritance

```
class MyParentClass:
    def __init__(self, x):
        self.x = x
    ...

class MyChildClass(MyParentClass):
    def __init__(self, x):
        super().__init__(x)
    ...
```

Pandas Cheatsheet

Data Processing

1. Import Pandas

```
import pandas as pd
```

2. Creating a Series

```
s = pd.Series([10, 20, 30, 40],
              index=['a', 'b', 'c', 'd'])
```

	0
a	10
b	20
c	30
d	40

3. Creating a DataFrame

```
df = pd.DataFrame({'Age': [28, 24, 35, 28],
                  'City': ['York', 'Oslo', 'Lima', 'York']},
                  index = ['John', 'An', 'Peter', 'Tom'])
```

	Age	City
John	28	York
An	24	Oslo
Peter	35	Lima
Tom	28	York

4. Column Operations: Adding, Multiplying

```
# Adding a new column
```

```
df['Salary'] = [50000, 60000, 75000, 55000]
```

```
# Compute new column
```

```
df['Bonus'] = df['Salary'] * 0.10
```

```
# Adding, subtracting, multiplying columns
```

```
df['Total'] = df['Salary'] + df['Bonus']
```

```
df['Years Until Retirement'] = 65 - df['Age']
```

5. Reading and Writing

```
df = pd.read_csv('data.csv')
df.to_csv('output.csv', index=False)
```

6. Accessing Data (Indexing and Slicing)

```
# Accessing rows:
```

```
df.iloc[3] # Access by position
```

```
df.loc['Peter'] # Access by index label
```

```
# Multiple indices
```

```
df.iloc[[2, 3]]
```

```
df.loc[['An', 'Peter']]
```

```
# Slicing rows
```

```
df.iloc[0:3:2] # Rows 0 to 3 (steps of 2)
```

```
# Accessing columns
```

```
df['Age'] # Single column -> Series
```

```
df[['City', 'Age']] # Multiple columns -> DF
```

7. Statistics

```
df['Age'].mean()
```

```
df['Age'].min()
```

```
df['Age'].median()
```

```
df['Age'].max()
```

```
df['Age'].sum()
```

```
df['Age'].std()
```

```
df['Age'].count() # Count non-null values
```

8. Filtering Data

```
# Filter rows based on a single condition.
```

```
df[df['Age'] > 30]
```

```
# Combine multiple conditions using & and |
```

```
filtered_df = df[
    (df['Age'] > 30) & (df['City'] == 'Lima')]
```

```
filtered_df = df[
```

```
    (df['Age'] < 30) | (df['City'] == 'York')]
```

9. Grouping

```
grouped = df.groupby('City')
```

```
# Loop over groups
```

```
for city, group in grouped:
```

```
    print(f"\nCity: {city}")
```

```
    print(group)
```

10. Aggregation Functions

```
grouped['Age'].mean()
```

```
grouped['Age'].min()
```

```
grouped['Age'].median()
```

```
grouped['Age'].max()
```

```
grouped['Age'].sum()
```

```
grouped['Age'].std()
```

```
grouped['Age'].count() # Count non-null values
```

```
grouped.head(n) # First n rows per group
```

11. Sorting Data

```
# Sort by a single column
```

```
df.sort_values(by='Age')
```

```
# Sort by multiple columns
```

```
df.sort_values(by=['Age', 'Salary'],
               ascending=[True, False])
```

12. Combining DataFrames

```
# Combine horizontally
```

```
combined = pd.merge(df1, df2, on='City',
                    how='left') # 'inner'/'outer'/'left'/'right'
```

```
# Combine vertically
```

```
combined = pd.concat([df1, df2])
```
